Five Memory Policies in a 5M Transformer: A Mac Smoke Sweep

Vuk Rosic
Independent Research Project

May 25, 2026

Abstract

We compare six attention styles in the same small transformer scaffold: dense attention, age-based forgetting, usage-refresh memory, competition-based memory, hierarchical summarization, and predictive importance memory. The goal here is not to claim a final winner. It is to verify that the five memory philosophies can be implemented, trained, and instrumented in one shared code path on a MacBook, while still producing readable plots and tables for later NVIDIA runs. In this smoke sweep, validation loss is essentially tied across the memory policies, while dense attention is much faster in plain PyTorch.

1 What We Are Testing

The research question is simple:

> If we keep the model and data fixed, do five different memory philosophies produce meaningfully different learning dynamics?

The five policies are:

- age-based forgetting,
- usage-refresh memory,
- competition-based memory,
- hierarchical summarization memory,
- predictive importance memory.

Dense attention is the baseline. It gives every token access to the full causal past. The memory policies instead keep a local window and route older context through compressed or scored memory blocks.

This report is a smoke test, not the final paper claim. The real purpose is to show that the code path is stable, the outputs are legible, and the same framework can later be moved to a longer NVIDIA run.

Table 1: Shared setup for the Mac smoke sweep.

Item	Value
Model preset	5M-parameter transformer
Hidden size	256
Layers / heads	6 / 8
Context window	256
Batch size	4
Training budget	2048 tokens
Seeds	1 (Mac smoke)
Dataset	synthetic causal copy-lag
Hardware	MacBook / MPS debug run
Main metric	validation loss

2 Setup

Table 1 gives the shared configuration. The model is the compact 5M-parameter preset discussed earlier in the repo, with a 256-token context window. For this Mac sweep, we only train for 2048 tokens so the run finishes quickly and the plots are easy to inspect.

The fairness rule is:

> Same model, same data, same batch size, same context length, same optimizer settings, and the same tiny training budget.

The only thing that changes is attention policy.

For the memory policies, we also track cheap post-run probe metrics. That keeps the training loop lightweight and pushes the extra instrumentation into a separate pass.

3 Results

Table 2 summarizes the sweep.

The main visual result is that the validation losses are almost indistinguish- able on this tiny run, while throughput separates the dense baseline from the memory policies very clearly. To make that easier to read, we use a single summary figure with loss deltas and throughput ratios instead of two nearly identical line charts.

Table 2: Mac smoke results. Peak CUDA memory is not meaningful on this MPS/CPU debug run, so the table emphasizes loss, throughput, and the probe metrics.

Policy	Loss	Tok/s	Vec.	Cov.	Gate	Refresh	Util.	Pred.	Sel.	Lvl.
dense	8.3776	1809.5	256	256	–	–	–	–	–	–
age-based forgetting	8.3772	166.9	80	128	0.3792	–	–	–	1.88	–
usage-refresh	8.3772	161.6	80	128	0.4204	0.1250	–	–	1.88	–
competition	8.3772	159.7	80	128	–	–	-2.4697	–	1.44	–
hierarchical	8.3772	159.0	84	128	0.9167	–	–	–	2.25	2.0
predictive	8.3798	148.6	80	128	–	–	–	0.5001	1.44	–

4 Interpretation

The smoke run says three useful things.

- The shared code path works. All six policies train and emit metrics.
- The memory policies are visually distinct. The probe plots are not the same shape, which is exactly what we wanted from the new abstractions.
- This tiny run is not enough to claim a real quality win. Dense is still the throughput leader, and the validation losses are almost tied.

That is a good result for a tutorial pipeline. It means the experiment is usable, not yet persuasive.

5 What This Does Not Prove

This run does *not* prove that any memory philosophy is better than dense. It only proves that we can compare them in a consistent harness and capture the behavior in plots and tables.

The next real step is the same comparison on NVIDIA, with more tokens and a few seeds. Then the question becomes interesting in the research sense:

> Do the five memory philosophies separate once the model has enough data and enough runtime?

6 Conclusion

We now have a paper-ready path for the new memory work:

> shared model → five memory policies → one fairness rule → automated plots and tables → PDF report.

On the Mac smoke run, the policies are distinguishable and the reporting is in place, but the losses are still too close to support a substantive claim. That is exactly the kind of honest result a mini research paper should report.

References

[1] DeepSeek-AI. *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*. 2026.

[2] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. 2019.

Figure 1: Policy comparison summary. Top-left: absolute validation loss, zoomed to the narrow range where the policies differ. Top-right: loss delta relative to dense, in milli-loss. Bottom-left: throughput on a log scale. Bottom-right: throughput relative to dense. This makes the near-ties in loss and the large throughput gap much easier to read at a glance.

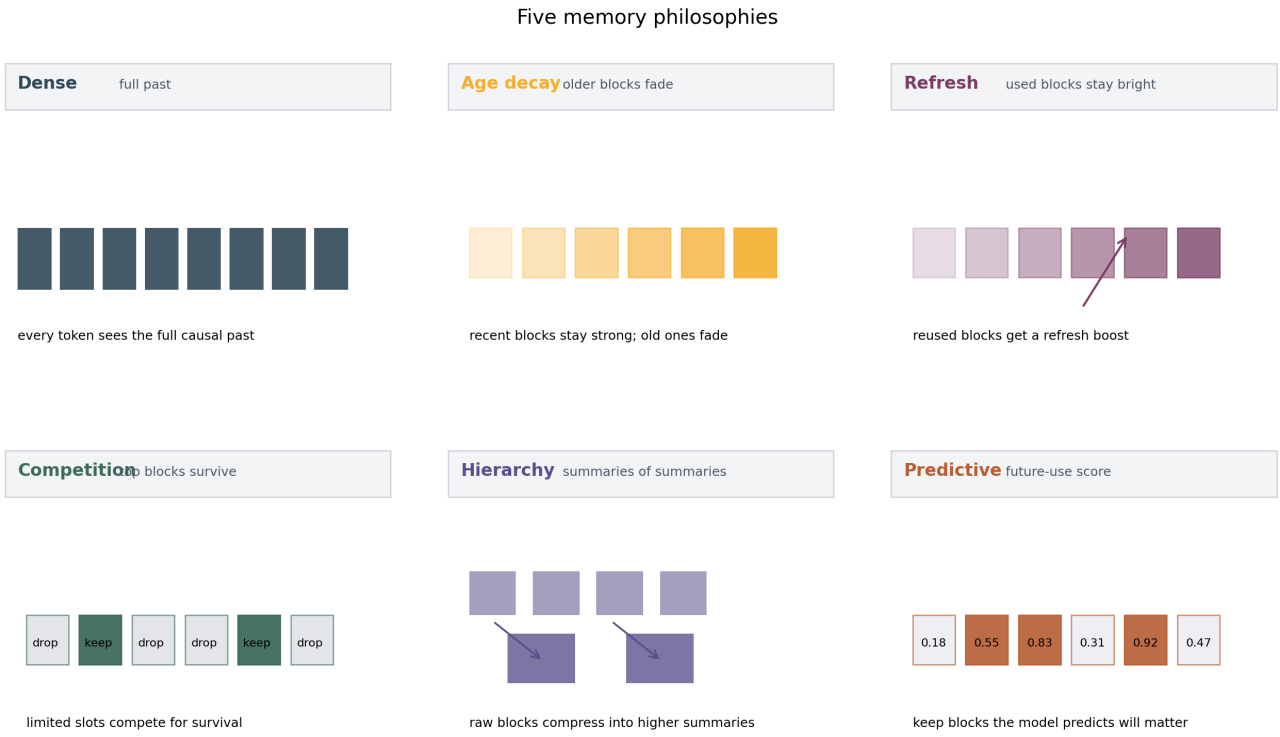

Figure 2: One-panel schematics for the five memory philosophies. Each panel shows the qualitative mechanism rather than the exact kernel implementation: age decay, refresh, competition, hierarchy, and predictive retention.

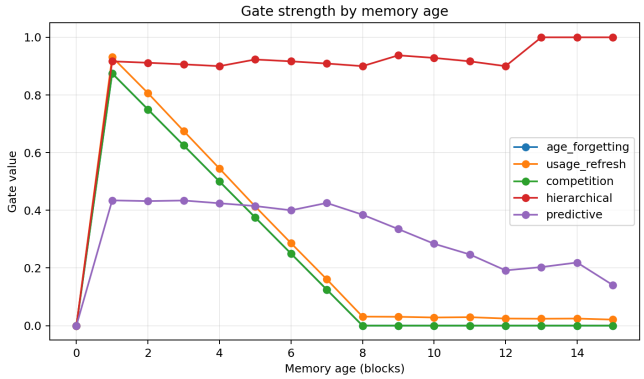

Figure 3: Age-based forgetting gate strength decreases with memory age. This is the one policy with a visibly monotonic retention curve in the smoke run.